

Portada

INDEX

1. INTRODUCCIÓN GENERAL DEL PROYECTO	4
2. INTRODUCCIÓN ESPECÍFICA DEL BACKEND	6
¿POR QUÉ ES TAN IMPORTANTE EL BACKEND?	7
TECNOLOGÍAS UTILIZADAS EN EL BACKEND	8
Node.js	8
Express	9
ARQUITECTURA DEL BACKEND	10
ESTRUCTURA DE CARPETAS DEL BACKEND	11
1. app.js	12
2. carpeta /routes	13
Responsabilitats:	13
Exemple:	13
Per què separar les routes?	13
3. carpeta /middleware	14
Exemple:	14
4. carpeta /controllers	15
Exemple:	15
Per què separar els controllers?	15
5. carpeta /services	16
6. carpeta /config	17
7. carpeta /tests	18
FLUJO INTERNO DEL BACKEND	19
RESPUESTA DEL BACKEND	21
Exemple:	21
CONCLUSIÓN DEL BACKEND	22
3. INTRODUCCIÓN ESPECÍFICA DEL FRONTEND	23
TECNOLOGÍAS UTILIZADAS EN EL FRONTEND	24
HTML	25
Exemple conceptual:	25
CSS	26
Exemple:	26
JavaScript	27
Exemple conceptual:	27
ESTRUCTURA DEL FRONTEND	28
index.html	29
Exemple:	29
carpeta /CSS	30
Fitxer principal:	30
Responsabilitat:	30
carpeta /JS	31
Fitxer principal:	31
S'encarrega de:	31
FUNCIONAMIENTO DEL FRONTEND	32

1. L'usuari obre l'aplicació	32
2. Introdueix dades	32
3. Prem "Analitzar"	32
4. Es recullen les dades	32
5. Es fa la petició al backend	32
6. El backend respon	32
7. Es mostra el resultat	32
CONCEPTOS TRABAJADOS	33
DOM	33
Esdeveniments (events)	33
Fetch	33
Gestió d'errors	33
Disseny responsive	33
CONCLUSIÓN DEL FRONTEND	34
FUTURES MILLORES	35
CONCLUSIONS	36

1. INTRODUCCIÓN GENERAL DEL PROYECTO

Actualment, internet s'ha convertit en una de les eines més utilitzades per milions de persones per comprar productes i contractar serveis. Cada dia es realitzen milers de transaccions en plataformes en línia, xarxes socials, marketplaces i pàgines web privades. Tot i que això ofereix molts avantatges, també presenta certs riscos.

Un dels problemes més freqüents és la dificultat de saber si un venedor és realment fiable. Moltes vegades els usuaris es troben amb anuncis falsos, pàgines web fraudulentos, preus massa baixos o venedors que desapareixen després de rebre el pagament.

Arran d'aquesta problemàtica sorgeix la idea d'aquest projecte: desenvolupar una aplicació web capaç d'analitzar la fiabilitat d'un venedor en línia utilitzant diferents criteris objectius.

La finalitat principal del sistema és oferir a l'usuari una eina senzilla que l'ajudi a prendre decisions més segures abans de fer una compra.

Aquest projecte consisteix en la creació d'una aplicació completa dividida en dues parts principals:

Backend

Frontend

El backend s'encarrega de processar la informació i aplicar la lògica de l'anàlisi.

El frontend s'encarrega de recollir dades de l'usuari i mostrar el resultat.

Inicialment, la idea del projecte era molt simple. L'usuari introduïa directament certes dades manualment, com ara si el venedor tenia telèfon, si tenia web o si tenia informes.

Tanmateix, aquesta solució tenia una limitació important: l'usuari continuava fent part de la feina que hauria de fer l'aplicació.

Per això, el projecte va evolucionar cap a una versió més intel·ligent.

En aquesta nova versió:

L'usuari només introdueix dades reals:

- telèfon
- web
- preu

I el sistema analitza automàticament:

- si el telèfon sembla vàlid
- si la pàgina web sembla vàlida
- si el preu és sospitosament baix
- si el venedor pot ser fiable

Això converteix el projecte en una proposta més realista i més útil.

A més, durant el desenvolupament s'ha treballat amb conceptes fonamentals del desenvolupament web com ara:

- arquitectura client-servidor
- APIs REST
- validació de dades
- separació de responsabilitats
- gestió d'errors
- connexió entre frontend i backend

Aquest projecte no només busca crear una aplicació funcional, sinó també demostrar l'aprenentatge adquirit durant el desenvolupament.

A nivell acadèmic, aquest projecte permet demostrar coneixements en:

- programació backend
- programació frontend
- disseny d'interfícies
- estructuració de projectes
- depuració d'errors
- proves i validacions

Per tant, el resultat final és una eina funcional, escalable i amb possibilitats de millora futura.

2. INTRODUCCIÓ ESPECÍFICA DEL BACKEND

El backend és una de les parts més importants del projecte, ja que és el “cervell” de tota l’aplicació. Mentre que el frontend és la part visual amb la qual interactua l’usuari, el backend treballa de manera interna, processant tota la informació i prenent decisions.

En aquest projecte, el backend és l’encarregat de rebre les dades que introdueix l’usuari des de la interfície, analitzar-les, validar-les, aplicar la lògica de negoci i retornar una resposta clara i estructurada.

Dit de manera senzilla, el backend fa tota la feina “intel·ligent” del sistema.

Per exemple:

si un usuari introdueix:

- un número de telèfon
- una pàgina web
- un preu

el backend és capaç d’analitzar:

- si el telèfon sembla real o no
- si la web té una estructura vàlida
- si el preu sembla sospitosament baix
- quina puntuació donar al venedor
- quin missatge mostrar a l’usuari

Per tant, el backend no només emmagatzema o mou informació, sinó que la interpreta.

¿POR QUÉ ES TAN IMPORTANTE EL BACKEND?

El backend és important perquè centralitza tota la lògica de l'aplicació.

Si tota la lògica estigués al frontend:

- seria fàcil de manipular
- seria menys segur
- l'usuari podria alterar els resultats

En tenir tota la lògica al backend:

- es millora la seguretat
- es millora l'organització
- es millora l'escalabilitat
- es millora el manteniment

A més, el backend permet que en un futur es puguin afegir noves funcionalitats sense haver de refer tota l'aplicació.

Per exemple:

en el futur es podria afegir:

- base de dades
- sistema de login
- historial d'anàlisis
- intel·ligència artificial
- APIs externes

I tot això partiria de l'estructura ja creada.

TECNOLOGÍAS UTILIZADAS EN EL BACKEND

Per desenvolupar aquesta part del projecte s'han utilitzat principalment dues tecnologies:

Node.js

Node.js és un entorn d'execució que permet utilitzar JavaScript fora del navegador.

Normalment, JavaScript s'utilitza per programar pàgines web.

Amb Node.js es pot utilitzar per crear servidors.

Això té diversos avantatges:

- utilitzar el mateix llenguatge al frontend i al backend
- treballar més ràpid
- reutilitzar coneixements
- gran comunitat i moltes llibreries

En aquest projecte, Node.js s'utilitza per:

- iniciar el servidor
- escoltar peticions
- executar la lògica

Express

Express és un framework per a Node.js.

La seva funció és simplificar la creació de servidors i APIs.

Sense Express, el codi seria molt més llarg i complicat.

Amb Express es poden crear fàcilment:

- rutes
- middlewares
- respostes HTTP
- gestió d'errors

En aquest projecte, Express s'utilitza per:

- crear rutes
- processar peticions
- enviar respostes
- connectar middlewares

Exemple:

```
app.post('/analitzar-venedor')
```

Això permet crear un endpoint per analitzar venedors.

ARQUITECTURA DEL BACKEND

Una de les decisions més importants durant el desenvolupament ha estat dividir el backend per capes.

Això es coneix com a arquitectura per capes.

La idea principal és separar les responsabilitats.

És a dir:

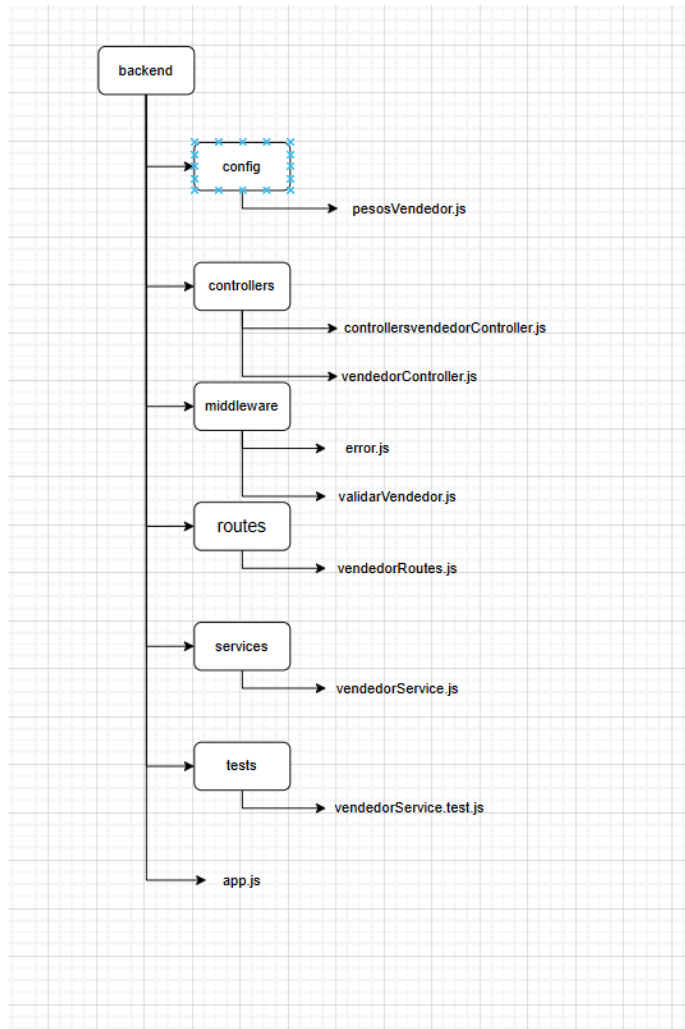
cada fitxer i cada carpeta fa una sola cosa.

Això fa que el projecte sigui:

- més net
- més fàcil d'entendre
- més fàcil de mantenir
- més fàcil d'ampliar

ESTRUCTURA DE CARPETAS DEL BACKEND

La estructura actual del backend es la siguiente:



Aquesta estructura no està feta al azar.
Cada carpeta té una funcionalitat concreta.

1. app.js

Aquest fitxer és el fitxer principal del backend.

És el punt d'entrada del servidor.

Quan executem `node app.js` al CMD, hem d'estar a la ruta de la carpeta on es troba el fitxer perquè aquest s'executi.

La seva funció és iniciar tota l'aplicació.

Dins d'aquest fitxer es fa:

- importar llibreries
- crear el servidor
- configurar middlewares
- carregar rutes
- gestionar errors
- escoltar peticions

2. carpeta /routes

La carpeta **routes** conté les rutes del projecte.

Les rutes són les portes d'entrada.

Quan el frontend envia una petició, per exemple:

POST /vendedor/analizar-vendedor

la ruta la rep.

La seva feina és decidir què fer amb ella.

Responsabilitats:

- aplicar middleware
- cridar el controller

Exemple:

`routes.post('/analizar-vendedor', validarVendedor, controller)`

Això significa:

- rep la petició
- valida les dades
- executa el controller

Per què separar les routes?

Perquè si tot estigués dins d'app.js:

- el fitxer seria enorme

Separar-ho fa que el codi sigui més net i organitzat.

3. carpeta /middleware

Els **middlewares** són funcions intermèdies.

S'executen abans del controller.

Serveixen per:

- validar dades
- bloquejar errors
- controlar accessos
- modificar peticions

En aquest projecte s'utilitzen per validar dades.

Exemple:

comprovar si:

- el telèfon existeix
- la web existeix
- el preu existeix
- els tipus són correctes

Si alguna cosa falla:

- atura l'execució

Si tot és correcte:

- executa next()

Això evita que arribin errors al service.

4. carpeta /controllers

Els **controllers** actuen com a intermediaris.

La seva feina és:

- rebre la petició i passar-la al service

No haurien de contenir lògica complexa.

Exemple:

```
const datos = req.body;  
const resultado = service.analizar(datos);  
res.json(resultado);
```

Per què separar els controllers?

Perquè així les rutes queden netes.

I perquè si demà canvies la lògica, no cal tocar la ruta.

5. carpeta /services

Aquesta és la part més important.

Aquí hi viu la lògica del programa.

En aquest projecte, aquí es fa:

- analitzar telèfon
- analitzar web
- detectar preu baix
- calcular punts
- generar raons
- interpretar el resultat

Aquí és on realment “pensa” l’aplicació.

Per exemple:

- si el venedor té telèfon → suma punts
- si té web → suma punts
- si el preu és sospitos → resta punts
- si té informes → resta punts

Després:

es genera un missatge:

- fiable
- dubtós
- no fiable

Aquesta carpeta està separada perquè la lògica pot créixer molt.

Si es barregés amb els controllers:

- el codi seria difícil de mantenir.

6. carpeta /config

Aquí es guarden configuracions.

Per exemple:

Els pesos: aquests són els punts que s'han de sumar o restar.

Exemple:

telèfon: +40

web: +40

preu molt baix: -25

Això permet canviar les regles sense tocar la lògica.

És molt útil per al manteniment.

7. carpeta /tests

Aquí hi anirien les proves.

Serveix per comprovar:

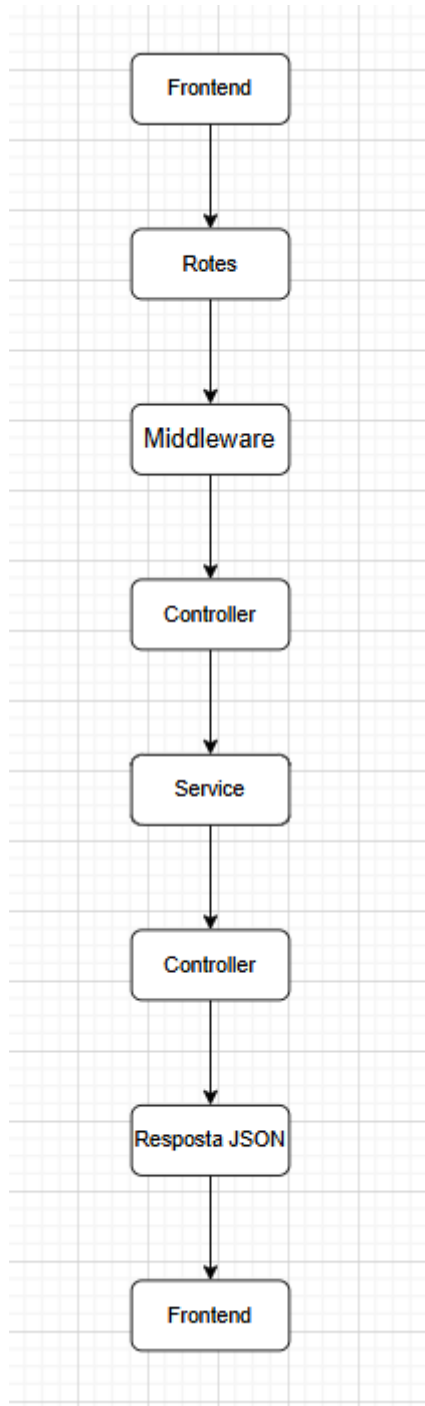
- rutes
- errors
- respostes

Permet automatitzar les proves.

Encara que no estigui completament desenvolupat, és important incloure-ho, ja que ajuda a la validació d'errors i és una forma de depuració.

FLUJO INTERNO DEL BACKEND

El flujo intern sería aixi:



Pas a pas:

- L'usuari envia dades.
- La ruta rep la petició.
- El middleware valida.
- El controller la rep.
- El service analitza.
- El service retorna el resultat.
- El controller respon.

RESPUESTA DEL BACKEND

La respuesta siempre intenta ser clara.

Exemple:

```
{
  "ok": true,
  "data": {
    "puntos": 80,
    "resultado": {
      "mensaje": "Vendedor bastante fiable",
      "color": "verde"
    },
    "razones": [
      "Tiene teléfono",
      "Tiene web"
    ]
  }
}
```

CONCLUSIÓN DEL BACKEND

El backend desenvolupat en aquest projecte no és només un servidor bàsic.

És una estructura organitzada, escalable i professional.

S'han aplicat conceptes reals de desenvolupament web:

- arquitectura per capes
- separació de responsabilitats
- validació de dades
- APIs REST

Gràcies a aquesta estructura, el projecte pot continuar creixent.

Per això, el backend és la base principal del sistema.

3. INTRODUCCIÓN ESPECÍFICA DEL FRONTEND

El frontend és la part visual del projecte. És la part amb la qual interactua directament l'usuari i, per tant, la part més visible de tota l'aplicació.

És la “cara” del sistema.

Mentre que el backend s'encarrega de realitzar tots els càlculs, validacions i anàlisis interns, el frontend té la missió de mostrar la informació a l'usuari i permetre-li interactuar amb l'aplicació de manera senzilla i intuïtiva.

Es pot dir que el frontend actua com un pont entre l'usuari i el backend.

La seva feina principal és:

- recollir informació de l'usuari
- enviar-la al backend
- rebre la resposta
- mostrar-la de manera clara

Sense frontend, l'aplicació continuaria funcionant internament, però no seria còmoda d'utilitzar. L'usuari hauria de fer peticions manuals des d'eines com Postman o des de la consola del navegador.

Per això, el frontend converteix aquest projecte en una aplicació real, accessible i usable.

A més, un frontend ben dissenyat millora l'experiència de l'usuari, fa l'aplicació més professional i facilita la comprensió dels resultats.

TECNOLOGÍAS UTILIZADAS EN EL FRONTEND

Per desenvolupar el frontend del projecte s'han utilitzat tres tecnologies principals:

- HTML
- CSS
- JavaScript

Aquestes tres tecnologies són la base del desenvolupament web.

Cadascuna té una funció diferent dins de l'aplicació.

HTML

HTML (HyperText Markup Language) és el llenguatge encarregat de crear l'estructura de la pàgina web.

És com l'esquelet de l'aplicació.

Amb HTML es defineixen:

- títols
- formularis
- botons
- caixes de text
- contenidors
- seccions

En aquest projecte, HTML s'utilitza per construir la interfície principal.

Per exemple:

- títol del projecte
- formulari d'entrada
- inputs
- botó d'analitzar
- contenidor del resultat

Sense HTML no existiria l'estructura visual.

Exemple conceptual:

```
<h1>Analitzador de venedor</h1>
```

Això crea el títol principal.

Un altre exemple:

```
<input type="text">
```

Això permet introduir dades.

HTML és simple però fonamental.

CSS

CSS (Cascading Style Sheets) és el llenguatge encarregat del disseny visual.

Si HTML és l'esquelet, CSS seria la roba o l'aparença.

Amb CSS es controla:

- colors
- mides
- marges
- alineació
- ombres
- animacions
- disseny responsive

En aquest projecte, CSS s'utilitza per:

- centrar el formulari
- dissenyar el botó
- millorar l'estètica
- mostrar els resultats de manera més clara
- adaptar la pàgina a diferents pantalles

Exemple:

```
background-color: blue;
```

Això canvia el color.

Un altre exemple:

```
display: flex;
```

Això permet organitzar elements.

Un bon CSS millora molt l'experiència de l'usuari.

Fa que l'aplicació sembli més professional.

JavaScript

JavaScript és el llenguatge que aporta funcionalitat i interactivitat.

És la part lògica del frontend.

Amb JavaScript es pot:

- detectar clics
- recollir dades
- fer peticions
- modificar HTML
- mostrar resultats
- mostrar errors

En aquest projecte, JavaScript s'encarrega de:

- escoltar quan l'usuari prem "Analitzar"
- recollir les dades introduïdes
- enviar-les al backend
- rebre la resposta
- mostrar l'anàlisi

Exemple conceptual:

```
button.addEventListener("click")
```

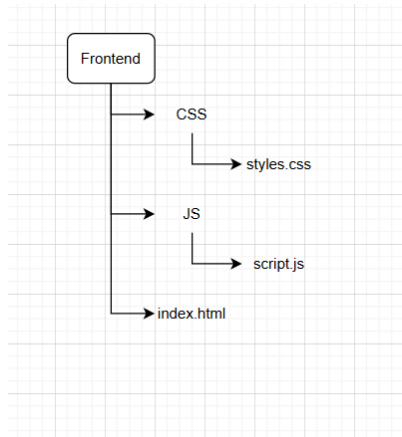
Això detecta el clic.

JavaScript és la part que "dona vida" al frontend.

ESTRUCTURA DEL FRONTEND

La estructura actual del frontend está organizada en carpetas.

Exemple:



Aquesta organizació ajuda a mantenir el projecte net.

Cada arxiu té una responsabilitat.

index.html

És el fitxer principal.

És la pàgina que veu l'usuari.

Conté:

- estructura
- formulari
- inputs
- botó
- contenidor del resultat

També connecta amb CSS i JS.

Exemple:

```
<link rel="stylesheet" href="CSS/styles.css">
```

i:

```
<script src="JS/script.js"></script>
```

Sense aquest fitxer no existiria la interfície.

carpeta /CSS

Aquí es guarda tot el disseny.

Fitxer principal:

`styles.css`

Responsabilitat:

- colors
- mides
- disseny
- responsive
- animacions

Es separa per no barrejar el disseny amb l'estructura.

Això fa que el projecte sigui més net.

carpeta /JS

Aquí es guarda la lògica.

Fitxer principal:

`script.js`

S'encarrega de:

- capturar dades
- detectar esdeveniments
- fer fetch
- mostrar resultats
- mostrar errors

Es separa per no barrejar la lògica amb l'HTML.

Això facilita el manteniment.

FUNCIONAMIENTO DEL FRONTEND

El flux del frontend és el següent:

1. L'usuari obre l'aplicació

2. Introdueix dades

Les dades:

- telèfon
- web
- preu

3. Prem "Analitzar"

JavaScript detecta l'esdeveniment.

```
form.addEventListener("submit")
```

4. Es recullen les dades

```
document.getElementById("telefono").value
```

5. Es fa la petició al backend

Amb fetch.

```
fetch("http://localhost:3000/vendedor/analizar-vendedor")
```

S'envien dades en format JSON.

6. El backend respon

La resposta conté:

- punts
- missatge
- raons

7. Es mostra el resultat

JavaScript modifica el DOM.

CONCEPTOS TRABAJADOS

Durant el desenvolupament del frontend s'han treballat conceptes importants:

DOM

Manipular l'HTML desde JavaScript.

Esdeveniments (events)

Detectar clics i submits.

Fetch

Enviar peticions HTTP.

Gestió d'errors

Mostrar missatges d'error.

Disseny responsive

Adaptar la pàgina a mòbil i ordinador.

CONCLUSIÓN DEL FRONTEND

El frontend desenvolupat en aquest projecte no és només una pantalla simple.

És una interfície funcional, visual i connectada amb el backend.

Permet que l'usuari utilitzi l'aplicació de manera fàcil.

Gràcies al frontend:

- es recullen dades
- es mostren resultats
- es millora l'experiència

Per això, el frontend és una part essencial del sistema.

FUTURES MILLORES

CONCLUSIONS